

.utility class

Benefits

By moving additional methods to a separate extension class (wrapper or subclass), you avoid gumming up client classes with code that doesn't fit. Program components are more coherent and are more reusable

How to Refactor

:Create a new extension class

.Option A: Make it a child of the utility class

Option B: If you have decided to make a wrapper, create a field in it for storing the utility class object to which delegation will be made. When using this option, you will need to also create methods that repeat the public methods of the utility class and contain simple delegation to the methods of the utility object

Create a constructor that uses the parameters of the constructor of the utility class

Also create an alternative "converting" constructor that takes only the object of the original class in its parameters. This will help to substitute the extension for the objects of the original class

Create new extended methods in the class. Move foreign methods from other classes to this class or else delete the foreign methods if their functionality is already present in the extension

Replace use of the utility class with the new extension class in places where its functionality is needed

Introduce Local Extension

مشکل (Problem)

یه کلاس کاربردی متدهایی که نیاز داری رو نداره. ولی نمیتونی این متدها رو به کلاس اضافه کنی.

راه حل (Solution)

یه کلاس جدید بساز که شامل متدها باشه و اون رو یا فرزند (Subclass) یا Wrapper کلاس کاربردی کن.

چرا ریفکتور کنیم؟ (Why Refactor)

توضیح	راه
از کلاس اصلی ارث‌بری کن، متدهای جدید رو اضافه کن. آسون‌تره، ولی اگه کلاس sealed باشه ممکن نیست	Subclass
یه کلاس جدید که متدهای جدید رو داره و بقیه رو به شیء اصلی تفویض می‌کنه. کارش بیشتره	Wrapper

مزایا (Benefits)

با انتقال متدهای اضافی به یه کلاس Extension جدا، کلاس‌های کلاینت رو با کد نامربوط شلوغ نمی‌کنی. کامپوننت‌های برنامه منسجم‌تر و قابل استفاده مجدد میشن.

مثال

قبل (نیاز به متد جدید روی DateTime):

csharp

// همه جا این کد تکرار شده

```
bool isWeekend = date.DayOfWeek == DayOfWeek.Saturday  
;date.DayOfWeek == DayOfWeek.Sunday ||
```

بعد با Wrapper:

csharp

```

public class EnhancedDateTime
{
    ;private DateTime _date

    ;public EnhancedDateTime(DateTime date) => _date = date

    // متد جدید
    ()public bool IsWeekend
    }

    return _date.DayOfWeek == DayOfWeek.Saturday
    ;date.DayOfWeek == DayOfWeek.Sunday_ ||

    {

        // متدهای اصلی DateTime (تفویض)
        ;public DateTime AddDays(int days) => _date.AddDays(days)
        ;public int Year => _date.Year
        ;public int Month => _date.Month
        // ... کلی متد تفویضی دیگر
    }

```

How to Refactor (چطور انجام بدیم)

توضیح	قدم
یه کلاس Extension جدید بساز: (A) زیرکلاس، یا (B) Wrapper با فیلد + متدهای تفویضی	۱
سازندهای بساز که پارامترهای سازنده کلاس اصلی رو بگیره	۲
یه سازنده تبدیلی هم بساز که فقط شیء کلاس اصلی رو بگیره	۳
متدهای جدید رو توی این کلاس بساز. Foreign Method ها رو از کلاسهای دیگه به اینجا منتقل کن	۴
استفاده از کلاس اصلی رو با کلاس Extension جدید جایگزین کن	۵

تمرین

این کد رو با Introduce Local Extension (Wrapper) اصلاح کن:

csharp

```

public class ReportService
{
    public void GenerateReport(string text)
    {
        // نیاز به مندی که تعداد کلمات رو بشمره
        ;int wordCount = text.Split(' ', StringSplitOptions.RemoveEmptyEntries).Length
        ;Console.WriteLine($"Word count: {wordCount}")
    }

    public void ValidateInput(string text)
    {
        // بازم همون کد تکراری!
        ;int wordCount = text.Split(' ', StringSplitOptions.RemoveEmptyEntries).Length
        if (wordCount < 10)
            ;Console.WriteLine("متن خیلی کوتاهه");
    }
}

```

راهنمایی: به کلاس EnhancedString (Wrapper دور string) با متد WordCount بساز.

بفرست تا بررسی کنیم! 

خودت حلش کن

کد اصلاح شده با Introduce Local Extension (Wrapper)

csharp

```

string دور Wrapper //
public class EnhancedString
{
    ;private string _value

    ;public EnhancedString(string value) => _value = value
}

```

```

// متد جدید
    ()public int WordCount
    }

;return _value.Split(' ', StringSplitOptions.RemoveEmptyEntries).Length
    {

// متدهای تفویضی برای دسترسی به string اصلی
    ;public string Value => _value
    ;public int Length => _value.Length

// سازنده تبدیلی
    ;public static implicit operator EnhancedString(string s) => new(s)
    ;public static implicit operator string(EnhancedString s) => s._value
    {

        public class ReportService
        {

            public void GenerateReport(string text)
            {

                ;var enhanced = new EnhancedString(text)
                ;Console.WriteLine($"Word count: {enhanced.WordCount()}")
            {

                public void ValidateInput(string text)
                {

                    ;var enhanced = new EnhancedString(text)
                    if (enhanced.WordCount() < 10)
                        ;Console.WriteLine("متن خیلی کوتاهه")
                {
            {

```

مقایسه

نسخه اصلاح	کد اصلی	
✓ WordCount() فقط یه جا	Split(...).Length تکراری	تکرار منطق
✓ فقط WordCount رو تغییر بده	باید همه جا رو عوض کنی	تغییر منطق
✓ enhanced.WordCount()	text.Split(...)	خوانایی

Introduce Local Extension تموم شد! 🎉

و با این، **Incomplete Library Class** بسته شد.

تبریک نهایی! 🏆

همه Code Smell ها و تمام تکنیک های ریفکتورینگ رو کار کردیم! 🍷🍷

خلاصه مسیر:

- Bloaters ✓
- Object-Orientation Abusers ✓
- Change Preventers ✓
- Dispensables ✓
- Couplers ✓
- Incomplete Library Class ✓

آفرین بر تو که تا آخرش با دقت و حوصله اومدی! 🙌🙌